

Data Cleaning and Data Engineering Guide

Smart Grocery Navigation and Scan-to-Pay Mobile Application

This guide explains the full data preparation work completed for the thesis project: how raw supermarket data was cleaned, validated, transformed, and converted into feature tables for machine learning, dashboards, backend APIs, and frontend integration.

Important academic note: the current dataset is a synthetic UrbanMart-style Nepali supermarket dataset. If your teacher later provides real Bhatbhateni data, the same pipeline can be reused, but the thesis should clearly state which dataset version was used.

Generated on: 2026-06-10 10:58

1. Project Data Pipeline Overview

The project follows a standard data science and data engineering workflow. Raw data is never edited directly. Instead, Python scripts read from raw files, produce cleaned files, and then generate machine-learning-ready feature tables.

```
data/raw
-> src/data_processing.py
-> data/processed
-> src/feature_engineering.py
-> data/features
-> ML models, backend APIs, dashboards, frontend app
```

This separation is important because it keeps the original dataset safe, makes the work reproducible, and allows anyone to rebuild the processed and engineered data whenever the raw data changes.

Main folders used

Folder	Purpose
data/raw	Original input dataset. These files are kept unchanged.
data/processed	Cleaned and standardized CSV files generated from raw data.
data/features	Data-engineering output: reusable feature tables for ML and application logic.
src/data_processing.py	Python script for data cleaning and quality validation.
src/feature_engineering.py	Python script for joining data and creating feature tables.
reports	Documentation and learning reports such as this PDF.

2. Raw Dataset Understanding

Before cleaning, the dataset was inspected to understand the available tables, their columns, relationships, row counts, and potential data quality problems. The raw dataset contains core supermarket operations: stores, aisles, products, customers, transactions, scanned items, app events, and payment events.

Raw file	Meaning
stores.csv	Store metadata such as city, area, location, size, and opening year.
zones.csv	Store zones such as fresh produce, dairy, frozen, snacks, and household areas.
aisles.csv	Aisle-level layout with x/y coordinates for navigation.
products.csv	Product catalog with SKU, price, shelf, aisle, zone, and store.
customers.csv	Customer profiles with loyalty tier, age bucket, home area, and app usage.
transactions.csv	Receipt-level transaction records. One row equals one shopping trip or purchase.
transaction_items.csv	Line items and scan events. This is the core basket and scan-to-pay log.
app_events.csv	Search and direction events from the mobile app.
payment_events.csv	Payment attempts, latency, retries, failures, and success records.

3. Data Cleaning: What Was Done

Data cleaning prepares the raw dataset so it can be trusted for analysis and engineering. In this project, cleaning was not just about removing blanks. It also included standardizing data types, checking relationships, reconciling mismatched summaries, and making the data consistent across tables.

Cleaning actions applied

- Column names were stripped of extra spaces so every table has stable column labels.
- Text fields were standardized by trimming whitespace and collapsing repeated spaces.
- Date and timestamp columns were parsed into real datetime values, then exported in a consistent format.
- Boolean fields such as `uses_app`, `scan_to_pay_used`, `is_festival_period`, `scan_success`, and `is_perimeter` were normalized.
- Duplicate primary keys were removed if they appeared, keeping the first valid record.
- Expected blanks in `app_events` were preserved during cleaning, then later converted to placeholders in feature engineering.
- Product shelf slot collisions were resolved so products do not occupy duplicated shelf coordinates in the processed layer.
- Transaction app search and direction counts were recalculated from `app_events`.
- Payment amounts were reconciled with transaction grand totals so payment events and transaction totals match.
- Foreign-key relationships were validated across stores, zones, aisles, products, customers, transactions, items, app events, and payment events.

Why `app_events` originally had blanks

The `app_events` table has two event types: `search_query` and `directions_request`. A search event naturally has a `search_query` but no `target_aisle_id`. A direction event naturally has a `target_aisle_id` but no `search_query`. These blanks were not data errors; they represented different event types. In the final feature layer, those blanks were converted into explicit values such as No Search Query and No Target Aisle so engineering and ML tools can process them more easily.

4. Data Cleaning Validation Results

The cleaning report records table size, duplicate rows, missing values, duplicate primary keys, date ranges, foreign-key violations, and pipeline corrections. The processed layer is the correct layer to use for analysis and engineering.

Dataset	Rows	Columns	Duplicates	Missing values	Corrections applied
stores.csv	8	9	0	0	0
zones.csv	17	4	0	0	0
aisles.csv	200	6	0	0	0
products.csv	1168	12	0	0	86
customers.csv	25000	7	0	0	0
transactions.csv	50000	22	0	0	695
transaction_items.csv	652655	15	0	0	0
app_events.csv	158241	8	0	158241	0
payment_events.csv	52879	8	0	0	2984
train_transaction_ids.csv	35000	1	0	0	0
val_transaction_ids.csv	7500	1	0	0	0
test_transaction_ids.csv	7500	1	0	0	0

In the processed layer, foreign-key violations are zero. This means the major relationships are valid: transaction items point to real transactions and products, products point to real aisles/zones/stores, and payment/app events point to real transactions.

5. Data Engineering: What It Means

Data engineering starts after cleaning. The goal is to convert clean operational data into structured feature tables. A feature table is a dataset where each row has a clear meaning and each column is useful for analytics, ML, dashboards, or app features.

For example, raw `transaction_items` contains individual purchased items. That is useful, but ML models usually need summarized behavior too: total basket size, total spend, scan failure rate, payment retries, product demand by day, and customer buying behavior. Data engineering creates those reusable summaries.

Feature table grain

Feature table	Grain	Main use
<code>transaction_features.csv</code>	One row per transaction	Checkout efficiency, payment behavior, basket analysis, anomaly detection.
<code>customer_features.csv</code>	One row per customer	Customer segmentation, personalization, loyalty analysis.
<code>product_features.csv</code>	One row per product	Product analytics, recommendation, scan issue detection.
<code>daily_store_product_features.csv</code>	One row per product per store per day	Demand forecasting, stock planning, inventory prediction.
<code>navigation_features.csv</code>	One row per app event	Search behavior, aisle navigation, product location analytics.

6. Engineered Feature Tables

The feature-engineering script generated five main feature tables and a report. These tables are now the best starting point for ML, backend APIs, dashboards, and frontend prototypes.

Feature table	Rows	Columns	Duplicates	Missing values
<code>transaction_features.csv</code>	50000	61	0	0
<code>customer_features.csv</code>	25000	32	0	0
<code>product_features.csv</code>	1168	35	0	0
<code>daily_store_product_features.csv</code>	958928	19	0	0
<code>navigation_features.csv</code>	158241	27	0	0

`transaction_features.csv`

This table summarizes each shopping trip. It combines transaction headers, scanned item behavior, payment behavior, customer profile, and store details. It is useful for measuring checkout efficiency and detecting complicated or risky transactions.

- Examples: `basket_size`, `total_units`, `grand_total_npr`, `dwell_minutes`, `unique_aisles`, `avg_scan_duration_ms`.
- Scan features: `total_rescans`, `failed_scan_items`, `scan_failure_rate`, `app_scan_share`.
- Payment features: `payment_attempts`, `retry_events`, `avg_payment_latency_ms`, `payment_success_flag`.
- Time features: `transaction_date`, `transaction_hour`, `transaction_dayofweek`, `is_weekend`.

- Business feature: `checkout_complexity_score`, which combines basket, aisle, payment, and rescan complexity.

customer_features.csv

This table summarizes customer-level behavior. It is designed for customer segmentation and personalized recommendations. Customers with no purchase history are still kept in the table, with explicit placeholder values, so the frontend and ML pipeline do not break.

- Examples: `transaction_count`, `total_spend_npr`, `avg_order_value_npr`, `total_units`.
- Engagement features: `scan_to_pay_rate`, `app_searches_total`, `app_directions_total`, `app_engagement_events`.
- Preference features: `preferred_store_id`, `preferred_payment_method`, `favorite_subcategory`, `favorite_zone_id`.
- RFM-style features: `recency_days`, `transaction_count`, `total_spend_npr`.
- Lifecycle feature: `customer_tenure_days` from registration date to latest transaction date.

product_features.csv

This table summarizes each product across all transactions. It is useful for product recommendations, inventory insight, scan reliability analysis, and product-level dashboard cards.

- Examples: `units_sold`, `sales_revenue_npr`, `transaction_count`, `unique_customers`.
- Location features: `zone_id`, `aisle_id`, `shelf_number`, `shelf_position`, `x_coord`, `y_coord`, `zone_name`.
- Performance features: `revenue_per_transaction_npr`, `units_per_transaction`, `festival_unit_share`.
- Scan features: `avg_scan_duration_ms`, `total_rescans`, `failed_scan_count`, `scan_failure_rate`.

daily_store_product_features.csv

This is the most important table for demand forecasting and inventory planning. It creates a complete day-by-day record for every product, including days with zero sales. This prevents forecasting models from only learning from days where sales happened.

- Examples: `sales_date`, `store_id`, `product_id`, `units_sold`, `revenue_npr`, `transaction_count`.
- Calendar features: `day_of_week`, `month`, `is_weekend`, `is_festival_period`.
- Forecasting features: `units_lag_1d`, `units_rolling_7d`, `units_rolling_14d`.
- Target column: `target_next_day_units` for supervised demand prediction.

navigation_features.csv

This table supports product search, aisle guidance, and store navigation analytics. It combines app events with transaction context, store details, and target aisle coordinates.

- Examples: `event_type`, `search_query`, `target_aisle_id`, `target_x_coord`, `target_y_coord`.
- Event timing: `event_hour`, `event_dayofweek`, `minutes_after_entry`.
- Navigation flags: `has_search_query` and `has_target_aisle`.
- Store context: `store_name`, `city`, `store_area`.

7. How This Supports ML and AI

The engineered tables map directly to the AI/ML goals of the project. Instead of training models directly on messy raw logs, models can use clean, stable, explainable feature tables.

ML task	Recommended table	Target / output
Demand forecasting	daily_store_product_features.csv	Predict target_next_day_units for each product/store/date.
Customer segmentation	customer_features.csv	Group customers by spending, recency, app use, and shopping behavior.
Product recommendation	transaction_items plus product_features/customer_features	Recommend products based on basket history and product popularity.
Scan-to-pay anomaly detection	transaction_features.csv	Flag high rescan, slow scan, repeated payment, or failed checkout behavior.
Navigation optimization	navigation_features.csv	Understand search-to-aisle behavior and product-location friction.

8. How This Supports Backend and Frontend

The frontend should not directly read raw CSV files. A backend API should read from cleaned and engineered tables, then return structured responses to the mobile app or dashboard.

```
data/features
  -> backend/API layer
  -> frontend mobile app and manager dashboard
```

Example API endpoints:

```
/products
/products/search?q=rice
/navigation/product/{product_id}
/recommendations/customer/{customer_id}
/dashboard/transactions
/dashboard/inventory-demand
/dashboard/payment-performance
```

The frontend can use transaction_features for dashboard cards, product_features for product screens, navigation_features for store guidance, and daily_store_product_features for inventory charts.

9. Explanation Script for Presentation

You can explain the work using this simple structure:

```
First, I kept the raw data unchanged in data/raw.
Then I created a cleaning pipeline using Python in src/data_processing.py.
This pipeline standardized text, timestamps, booleans, duplicates, payment totals, app counts, and ta
The cleaned outputs were stored in data/processed.
```

```
After that, I created a data engineering pipeline in src/feature_engineering.py.
This joined the cleaned tables and created feature tables in data/features.
The feature tables have clear row meanings: one row per transaction, customer, product, product-day,
These feature tables are now ready for ML models, dashboards, backend APIs, and frontend features.
```

10. Final Folder Structure

```
Individual/
  data/
    raw/
      original CSV files
    processed/
      *_clean.csv
      cleaning_report.csv
    features/
      transaction_features.csv
      customer_features.csv
```

```

■      ■■■ product_features.csv
■      ■■■ daily_store_product_features.csv
■      ■■■ navigation_features.csv
■      ■■■ feature_engineering_report.csv
■■■ src/
■      ■■■ data_processing.py
■      ■■■ feature_engineering.py
■■■ notebooks/
■■■ models/
■■■ reports/
■      ■■■ data_cleaning_and_data_engineering_guide.pdf
■■■ README.md

```

11. Recommended Frontend Folder Structure

When you start frontend development, keep it separate from the data and ML pipeline. This makes the project cleaner and easier to explain.

```

frontend/
■■■ lib/
■      ■■■ main.dart
■      ■■■ app/
■      ■      ■■■ app.dart
■      ■      ■■■ routes.dart
■      ■      ■■■ theme.dart
■      ■■■ core/
■      ■      ■■■ api/
■      ■      ■■■ constants/
■      ■      ■■■ errors/
■      ■      ■■■ utils/
■      ■      ■■■ widgets/
■      ■■■ features/
■      ■      ■■■ auth/
■      ■      ■■■ home/
■      ■      ■■■ product_search/
■      ■      ■■■ store_navigation/
■      ■      ■■■ scan_to_pay/
■      ■      ■■■ cart/
■      ■      ■■■ payment/
■      ■      ■■■ order_history/
■      ■      ■■■ recommendations/
■      ■      ■■■ manager_dashboard/
■      ■■■ data/
■      ■      ■■■ models/
■      ■      ■■■ repositories/
■      ■      ■■■ services/
■■■ assets/
■      ■■■ images/
■      ■■■ icons/
■      ■■■ maps/
■■■ test/
■■■ pubspec.yaml

```

12. Key Takeaway

The most important point is that the project now has a reproducible data pipeline. Raw data is preserved, cleaned data is stored separately, and engineered features are generated for AI, dashboards, backend APIs, and frontend use. This is exactly the kind of structure expected in a strong final-year ML/AI project.